

RSLAQ DRL Implementation

Compatibility Analysis Report

Feasibility Analysis: Implementing RSLAQ Paper DRL
with `rslaq_env.py` and ns-3 Simulator

Reference: Yungaicela-Naula et al., "RSLAQ - A Robust SLA-driven
6G O-RAN QoS xApp Using Deep Reinforcement Learning", IEEE TMC 2026

Generated by Z.ai

April 2026

Table of Contents

1. Executive Summary	4
2. RSLAQ Paper: DRL Formulation	4
2.1 DRL Algorithm	4
2.2 State Space	5
2.3 Action Space	5
2.4 Reward Function	6
3. Current rslaq_env.py Analysis	6
3.1 Architecture Overview	6
3.2 Current State Space	7
3.3 Current Action Space	7
3.4 Current Reward Function	7
4. Detailed Gap Analysis	8
4.1 Metric Mapping: Paper vs ns-3	8
4.2 Critical Gaps Summary	8
5. Feasibility Assessment	9
5.1 What Works Well	9
5.2 Major Challenges	9
5.3 Feasibility Verdict	10

6. Implementation Roadmap	10
6.1 Phase 1: State Space Redesign (Effort: 3-5 days)	11
6.2 Phase 2: Action Space Conversion (Effort: 5-7 days)	11
6.3 Phase 3: Reward Function Overhaul (Effort: 5-7 days)	11
6.4 Phase 4: DRL Agent Implementation (Effort: 5-7 days)	11
6.5 Phase 5: Validation and Experimentation (Effort: 5-10 days)	11
7. Alternative Approach: Continuous-Action DRL	12
8. Conclusion	13

1. Executive Summary

This report presents a detailed feasibility analysis for implementing the Deep Reinforcement Learning (DRL) agent described in the RSLAQ paper using the existing `rs_laq_env.py` Gymnasium environment, with the ns-3 mmWave O-RAN simulator replacing the MATLAB-based simulator used in the original research. The analysis reveals that the current environment is a simplified prototype that captures the high-level concept of the paper but requires significant modifications across four critical dimensions to faithfully reproduce the DRL formulation: (1) the state representation, which currently uses only 12 dimensions instead of the 16 defined in the paper; (2) the action space, which uses continuous PRB allocation without the semi-dynamic slice isolation mechanism; (3) the reward function, which lacks the outage/soft SLA penalty structure with terminal state logic; and (4) the DRL algorithm itself, where the paper specifies Double Deep Q-Learning (DDQL) while the current environment is compatible with continuous-action algorithms like SAC or PPO.

The conclusion is that implementation is feasible but requires substantial refactoring. The ns-3 simulator provides the necessary metrics (throughput, delay, packet loss, buffer status) to construct both the paper-defined state and reward. However, some metrics defined in the paper (bytes to transmit, buffer status, resource share) require careful mapping from the ns-3 output data. The most critical adaptation needed is the redesign of the action space from continuous to discrete (198 possible actions) to match the DDQL algorithm, along with the implementation of the semi-dynamic resource allocation mechanism that splits PRBs into 50% static and 50% dynamic components.

2. RSLAQ Paper: DRL Formulation

2.1 DRL Algorithm

The RSLAQ paper employs Double Deep Q-Learning (DDQL) with experience replay as its core DRL algorithm. This choice is motivated by the need for simplicity and timely response in the xApp context, where decisions must be made every 10 ms (one 5G frame). DDQL uses two deep neural networks with identical structure: an Online DNN with parameters θ and a Target DNN with parameters $\hat{\theta}$. The Online DNN is trained at each learning step to minimize the loss function, while the Target DNN remains frozen to enhance learning stability. The Target DNN is periodically updated to match the Online DNN after a predefined number of steps ($nsut = 200$ in the selected hyperparameter set). The experience replay mechanism stores past transitions of the form $(s, a, r, s', tstate)$ where $tstate$ is a binary indicator for terminal states.

The neural network architecture consists of four 2D convolutional layers, each followed by batch normalization, and a fully connected output layer using Tanh activation. The epsilon-greedy method is used for exploration, with exponential decay ($\lambda_{\epsilon} = 0.998$). The selected hyperparameter set (Hyp-set 3) uses a learning rate of 0.001, discount factor $\gamma = 0.80$, batch size of 350, replay buffer capacity of

500, target network update every 200 steps, and episode reset every 100 steps. The agent trains for up to 3500 steps per scenario, where each step corresponds to one 10 ms frame.

2.2 State Space

The state representation in the paper is defined as a matrix s in $\mathbb{R}^{4 \times (J+1)}$, where J is the number of slices. For the 3-slice case study (eMBB, URLLC, MTC), this results in a state of $\mathbb{R}^{4 \times 4} = 16$ dimensions. The four metric rows represent: (1) btx - bytes to transmit per slice, (2) bfs - buffer status per slice, (3) rsh - resource share per slice, and (4) tdp - transmitted/dropped bytes per slice. The last column aggregates cell-level totals across all slices. These metrics were specifically chosen to reflect the dynamics of the most common slice types: btx and bfs help track eMBB and MTC performance, while bfs and tdp assist in monitoring URLLC performance. The state is gathered from the E2 node (DU) statistics every 10 ms.

Metric	Symbol	Description	Purpose
Bytes to transmit	btx	Pending bytes queued for transmission per slice	Track eMBB/MTC load
Buffer status	bfs	Percentage of buffer occupancy per slice	Track URLLC congestion
Resource share	rsh	Current PRB proportion allocated per slice	Current allocation state
Dropped bytes	tdp	Transmitted or dropped bytes per slice	Track packet loss

Table 1. State metrics defined in the RSLAQ paper (Equation 1)

2.3 Action Space

The action space in RSLAQ is discrete with $|A| = 198$ possible actions. Each action consists of two components: (1) the proportion of PRBs allocated to each of the J slices (p_j), and (2) the scheduler selection (sch) from {RR, PF, BCQI}. The PRB proportions are discretized to $P = \{0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1\}$ and must satisfy the constraint that the sum of proportions across all slices equals 1. A critical design element is the semi-dynamic slice isolation mechanism: 50% of PRBs are allocated statically (p_{sta}) based on operator-defined weights, while the remaining 50% are optimized dynamically (p_{opt}) by the DRL agent. This balances resource optimization with slice isolation, ensuring that every slice receives a minimum guaranteed allocation while still allowing the agent to adapt to changing network conditions.

Component	Paper Definition	Current Env
Type	Discrete (198 actions)	Continuous Box
PRB allocation	p_j in $\{0, 0.1, \dots, 1\}$, $\text{sum} = 1$	[dedicated, min, max] per slice
Scheduler selection	{RR, PF, BCQI}	Not implemented
Slice isolation	50% static + 50% dynamic	Not implemented

Component	Paper Definition	Current Env
Output format	$(\{p_1, \dots, p_j\}, \text{sch})$	$(\text{slice_idx}, \text{dedicated}, \text{min}, \text{max})$

Table 2. Comparison of action space definitions

2.4 Reward Function

The reward function in RSLAQ is the most complex component, consisting of three conditional branches based on SLA compliance. The optimization reward (r_{opt}) is a weighted sum of per-slice KPIs plus an action cost term. For eMBB, the optimization KPI h11 maximizes the average throughput per UE scaled by the slice weight. For URLLC, h12 uses an exponential transformation of the maximum buffer status to convert minimization into a positive value. For MTC, h13 maximizes average throughput. The action cost penalizes non-RR schedulers (cost = 1 for RR, cost = 2 for PF/BCQI) since RR does not require CQI reports and is less resource-intensive.

The SLA assurance component introduces outage conditions and soft conditions that trigger terminal states. An outage occurs when the SLA violation rate exceeds the threshold defined by $(1 - \text{reliability})$, and results in a negative reward of $-\sum(\phi_j * \omega_j)$. A soft SLA violation (e.g., eMBB throughput exceeding the maximum agreed limit) results in reward = 0 and a terminal state. This structure forces the DRL agent to prioritize SLA compliance during training, with episodes terminating whenever critical KPIs are violated. The paper assumes 100% reliability, meaning any deviation from SLA targets is treated as an outage.

Condition	Reward	Description
Soft SLA violation	0 (Terminal)	Any soft KPI exceeded; episode ends
Outage SLA violation	$-\sum(\phi_j * \omega_j)$ (Terminal)	Critical KPI violated; episode ends with penalty
Normal operation	$r_{\text{opt}} = \sum(\omega_j * h_j) + 1/\text{cost}(\alpha)$	Weighted optimization of per-slice KPIs

Table 3. Reward function branches (Equation 12)

3. Current rslaq_env.py Analysis

3.1 Architecture Overview

The current `rslaq_env.py` implements a Gymnasium environment that inherits from `Ns0ranEnv`, the base class for ns-3 O-RAN simulations. The environment manages the interaction loop with the ns-3 simulator through semaphores and subprocess control, reading KPM metrics from simulation output files (`rslaq-kpms.txt`) and writing control actions to a CSV file (`rslaq_actions_for_ns3.csv`). The environment uses a unique simulation UUID for each run and maintains an SQLite database (`datalake`) for storing metrics and

state information during the simulation. This architecture is well-suited for the intended purpose but currently implements a simplified version of the RSLAQ paper formulation.

3.2 Current State Space

The observation space is defined as a Box with shape (3, 4), yielding 12 dimensions. The three rows correspond to the three slices (eMBB, URLLC, MTC), while the four columns represent: (0) throughput in Mbps, (1) average delay in ms, (2) packet delivery ratio (PDR), and (3) number of lost packets. These metrics are aggregated from the ns-3 simulation output per UE and then averaged per slice. The observation values are stored in a numpy array and updated in the `_fill_datalake_usecase` method by parsing the `rsraq-kpms.txt` file.

The key differences from the paper are: (1) the paper uses 16 dimensions (4 metrics x 4 columns including cell totals) versus the current 12 (3 slices x 4 metrics); (2) the paper includes btx, bfs, rsh, and tdp metrics, while the current env uses throughput, delay, PDR, and lost_packets; (3) the paper includes a cell-level aggregate column, which is absent in the current implementation. While throughput and delay map relatively well to the paper concepts, buffer status (bfs), resource share (rsh), and bytes to transmit (btx) require different extraction logic from the ns-3 simulation data.

3.3 Current Action Space

The action space is defined as a Box with shape (3, 3) and bounds [0, 100], representing a continuous action space. Each slice receives three parameters: dedicated PRB percentage, minimum PRB percentage, and maximum PRB percentage. The `_compute_action` method enforces constraints such as `min_prb >= dedicated` and `max_prb >= min_prb`. The output format is a list of tuples (slice_idx, dedicated, min_prb, max_prb) sent to the ns-3 simulator.

This design is fundamentally different from the paper in several ways. First, the paper uses a discrete action space (198 actions) compatible with DDQL, while the current env uses a continuous space compatible with algorithms like SAC, PPO, or DDPG. Second, the paper includes scheduler selection (RR, PF, BCQI) as part of the action, which is not present in the current implementation. Third, the semi-dynamic resource allocation mechanism (50% static + 50% dynamic) is not implemented. Fourth, the current env sends absolute PRB percentages to the simulator, while the paper computes proportions that must sum to 1.

3.4 Current Reward Function

The reward function computes a weighted SLA compliance score across the three slices. For each slice, it checks whether throughput exceeds the minimum threshold, delay stays below the maximum threshold, and PDR exceeds the minimum threshold. Each satisfied condition contributes 1/3 to the slice score, which is then multiplied by the slice weight (`omega_j`). The weights are defined as 0.4 for URLLC, 0.4 for eMBB, and 0.2 for MTC. This yields a reward in the range [0, 1].

Compared to the paper, the current reward lacks several critical features: (1) no terminal state logic for outage or soft SLA violations; (2) no action cost component for scheduler selection; (3) no soft KPI violation handling (e.g., penalizing over-allocation to eMBB beyond the maximum agreed throughput); (4) the optimization component uses simple threshold-based compliance rather than the continuous KPI optimization defined in Equations 16-19; (5) the weights differ from the paper (paper uses [0.33, 0.40, 0.27] based on priorities [2, 1, 3]).

4. Detailed Gap Analysis

4.1 Metric Mapping: Paper vs ns-3

The following table maps each metric defined in the RSLAQ paper to the corresponding data available from the ns-3 simulation. The ns-3 rslaq-kpms.txt file provides per-UE statistics including rxBytes, txBytes, rxPackets, txPackets, lostPackets, throughputMbps, avgDelayMs, and pdr. Some paper metrics can be directly mapped, while others require computation or may not be directly available.

Paper Metric	Symbol	ns-3 Available?	Mapping Strategy
Throughput	thr	Yes (throughputMbps)	Direct mapping from UE stats
Buffer status	bfs	Partial (txBytes/rxPackets)	Compute as queue ratio from txBytes
Bytes to transmit	btx	Yes (txBytes)	Direct mapping, sum per slice
Dropped bytes	tdp	Yes (lostPackets)	Map from lostPackets per UE
Resource share	rsh	No (computed from action)	Track from previous action applied
Delay	delay	Yes (avgDelayMs)	Direct mapping, avg per slice
Packet delivery ratio	pdr	Yes (pdr)	Direct mapping

Table 4. Metric mapping between paper and ns-3 simulator output

4.2 Critical Gaps Summary

ID	Component	Gap Description	Impact	Effort
G1	State space	12 dims vs 16 dims; different metrics (throughput/delay vs btx/bfs/rsh/tdp)	High	Medium

ID	Component	Gap Description	Impact	Effort
G2	Action space	Continuous Box vs discrete 198 actions; no scheduler selection; no semi-dynamic isolation	High	High
G3	Reward function	No terminal states; no outage/soft penalties; no action cost; different weights	High	High
G4	DRL algorithm	DDQL required (discrete) vs continuous-compatible env; different NN architecture	Critical	High
G5	Scheduler control	No scheduler selection (RR/PF/BCQI) in action; ns-3 may need slice-aware scheduler	Medium	High
G6	Episode reset	No ntsr-based episode reset (every 100 steps) or terminal state handling	Medium	Low
G7	ns-3 metrics	rsh (resource share) not directly available; bfs (buffer status) requires computation	Medium	Medium

Table 5. Critical gaps between paper and current implementation

5. Feasibility Assessment

5.1 What Works Well

Several aspects of the current implementation provide a solid foundation for the paper reproduction. First, the **NsOranEnv** base class already handles the complex interaction with the ns-3 simulator, including subprocess management, semaphore-based synchronization, datalake storage, and control file generation. This infrastructure represents a significant amount of work that does not need to be redone. Second, the ns-3 `rsdaq-kpms.txt` file provides most of the raw metrics needed to construct the paper-defined state space, particularly throughput (`txBytes`), delay (`avgDelayMs`), and packet loss (`lostPackets`). Third, the control interface already supports sending PRB allocation commands to the simulator, which aligns with the paper action space for resource distribution.

Additionally, the Gymnasium API compliance of the current environment (`reset`, `step`, `observation_space`, `action_space`) means that standard DRL libraries like Stable-Baselines3 or Ray RLlib can be used for training. The ns-3 scenario configuration (`rsdaq-simulation-mac-slicing`) already supports 3 slices with configurable UEs per slice, matching the paper experimental setup. The simulation timestep of 10 ms is also compatible with the paper xApp decision interval.

5.2 Major Challenges

The most significant challenge is the fundamental mismatch between the discrete action space required by DDQL (198 actions) and the continuous action space currently implemented. Converting to discrete actions requires: (a) generating all valid action combinations (PRB proportions that sum to 1, multiplied by 11 scheduler options), (b) modifying the ns-3 control interface to accept scheduler type, and (c) rewriting the action post-processing logic. The semi-dynamic slice isolation adds further complexity, as the agent only controls 50% of the resources while the other 50% is statically allocated.

The second major challenge is the reward function redesign. Implementing the paper reward requires: (a) computing SLA violation rates (v_{rsla}) per KPI per slice using the empirical probability formula, (b) determining outage conditions (ϕ_j) and soft conditions (ρ_j) based on reliability thresholds, (c) implementing terminal state logic that resets the environment, (d) computing the optimization component with exponential transformation for URLLC buffer minimization, and (e) adding the action cost for scheduler selection. This reward structure is significantly more complex than the current threshold-based compliance score and is central to the RSLAQ robustness properties demonstrated in the paper experiments.

The third challenge involves the ns-3 simulator capabilities. The paper assumes a slice-aware scheduler that can distribute PRBs probabilistically according to xApp commands and select between RR, PF, and BCQI schedulers per slice. The current ns-3 simulation may need modifications to support scheduler selection and the probabilistic resource distribution described in the paper. Additionally, some metrics (bfs - buffer status as a percentage, rsh - resource share) may not be directly available from ns-3 and need to be computed or estimated from available data.

5.3 Feasibility Verdict

Question	Answer
Can the DRL be implemented with the current env?	Partially - requires major refactoring
Can ns-3 replace MATLAB simulator?	Yes - with metric mapping adaptations
Is DDQL compatible with current env?	No - action space must be converted to discrete
Can all paper metrics be obtained from ns-3?	Mostly - rsh and bfs need computation
Is the reward function reproducible?	Yes - with significant coding effort
Estimated effort for full paper reproduction?	2-4 weeks of dedicated development

Table 6. Feasibility verdict for each implementation dimension

6. Implementation Roadmap

The following roadmap outlines the steps needed to adapt the current environment to faithfully reproduce the RSLAQ paper DRL with the ns-3 simulator. Each phase addresses a specific set of gaps and builds upon the previous phase.

6.1 Phase 1: State Space Redesign (Effort: 3-5 days)

Modify the `observation_space` from `Box(3, 4)` to `Box(4, 4)` to match the paper state matrix. Replace the current four metrics (throughput, delay, pdr, lost_packets) with the paper-defined metrics (btx, bfs, rsh, tdp) and add the cell-level aggregate column. Update `_fill_datalake_usecase` to compute btx as sum of txBytes per slice, bfs as buffer occupancy ratio, rsh by tracking the last applied action proportions, and tdp as transmitted or dropped bytes. Normalize all metrics to ensure stable DRL training. Add cell-level totals as the fourth column in the state matrix.

6.2 Phase 2: Action Space Conversion (Effort: 5-7 days)

Convert the action space from continuous `Box(3, 3)` to discrete `Discrete(198)`. Generate all valid action combinations where: (a) PRB proportions p_j are selected from $\{0, 0.1, \dots, 1\}$ with $\text{sum} = 1$ across 3 slices; (b) scheduler `sch` is selected from $\{\text{RR}, \text{PF}, \text{BCQI}\}$; (c) the semi-dynamic mechanism applies $p_{\text{static}} = \omega_j * 0.5$ and $p_{\text{dynamic}} = \text{model_output} * 0.5$. Modify `_compute_action` to decode discrete action indices into $(p1, p2, p3, \text{sch})$ tuples and apply the slice isolation formula. Update the control file format to include scheduler type. This phase may also require ns-3 simulator modifications to accept scheduler selection commands.

6.3 Phase 3: Reward Function Overhaul (Effort: 5-7 days)

Implement the three-branch reward function from Equation 12. Compute SLA violation rates (v_{rsla}) using the empirical probability formula in Equation 10. Implement outage detection (ϕ_j) using Equation 11 with configurable reliability thresholds. Implement soft KPI violation detection (ρ_j) for eMBB maximum throughput. Compute the optimization reward r_{opt} using Equations 16-19 with proper normalization. Add terminal state logic to handle episode resets on SLA violations. Integrate the action cost for scheduler selection (Equation 9). Update weights to match paper priorities $[0.33, 0.40, 0.27]$.

6.4 Phase 4: DRL Agent Implementation (Effort: 5-7 days)

Implement the DDQL agent following Algorithm 1 in the paper. Build the neural network architecture with four 2D convolutional layers, batch normalization, and a fully connected output layer with Tanh activation. Implement experience replay buffer with capacity $L=500$ and batch size $\text{btsz}=350$. Configure epsilon-greedy exploration with exponential decay ($\lambda_{\text{epsilon}}=0.998$, $\text{epsilon_min}=0.05$). Set hyperparameters to Hyp-set 3: $\text{LR}=0.001$, $\gamma=0.80$, target update every 200 steps, episode reset every 100 steps. Implement the training loop with proper terminal state handling and target network synchronization.

6.5 Phase 5: Validation and Experimentation (Effort: 5-10 days)

Replicate the five experimental scenarios from the paper (Low traffic, Normal, Congestion, Stressed, Insufficient resources) by configuring the ns-3 simulation parameters. Compare RSLAQ performance against baseline schedulers (RR, PF, BCQI) and the SAC approach. Measure SLA compliance rates, throughput per slice, buffer status per slice, and resource utilization efficiency. Validate robustness properties under varying traffic conditions and SLA configurations. This phase may require iterative tuning of ns-3 simulation parameters to match the paper experimental conditions.

Phase	Focus Area	Gaps Addressed	Effort
1	State Space Redesign	G1, G7	3-5 days
2	Action Space Conversion	G2, G5	5-7 days
3	Reward Function Overhaul	G3, G6	5-7 days
4	DRL Agent Implementation	G4	5-7 days
5	Validation and Experimentation	All	5-10 days

Table 7. Implementation roadmap phases and estimated effort

7. Alternative Approach: Continuous-Action DRL

If the discrete action space of DDQL proves too restrictive or difficult to implement with the ns-3 control interface, an alternative approach is to use a continuous-action DRL algorithm such as Soft Actor-Critic (SAC) or Proximal Policy Optimization (PPO). This approach would maintain the current continuous Box action space and require fewer modifications to the environment. The paper itself evaluates SAC as a baseline and shows that RSLAQ (DDQL) outperforms it, but SAC still achieves reasonable results. Using SAC would significantly reduce the implementation effort (eliminating Phase 2 of the roadmap) and allow faster experimentation with the ns-3 simulator.

However, this alternative sacrifices several key features of the paper: (1) the scheduler selection component (SAC uses fixed PF in the paper baseline); (2) the semi-dynamic slice isolation mechanism; (3) the discrete action structure that enables exact reproducibility of paper results. If the goal is to faithfully reproduce the paper for academic validation, DDQL with discrete actions is strongly recommended. If the goal is to build a functional RL-based slicing system with ns-3, SAC with the current continuous action space provides a viable and faster path to results.

Aspect	DDQL (Paper Fidelity)	SAC (Pragmatic Alternative)
Action space	Discrete (198 actions)	Continuous (Box)

Aspect	DDQL (Paper Fidelity)	SAC (Pragmatic Alternative)
Scheduler selection	Yes (RR/PF/BCQI)	No (fixed PF)
Slice isolation	Semi-dynamic (50/50)	Not implemented
Paper reproducibility	High	Low-Medium
Implementation effort	3-4 weeks	1-2 weeks
ns-3 modifications	May need scheduler API	Minimal
DRL library support	Custom implementation	Stable-Baselines3 native

Table 8. Comparison between paper-faithful DDQL and pragmatic SAC approach

8. Conclusion

The implementation of the RSLAQ paper DRL agent using the existing `rsdaq_env.py` and ns-3 simulator is technically feasible but requires substantial modifications across multiple dimensions. The current environment provides a functional Gymnasium wrapper around ns-3 with basic slicing support, which serves as a valid starting point. However, to faithfully reproduce the paper results, the following modifications are essential: converting the state space to use paper-defined metrics (btx, bfs, rsh, tdp), restructuring the action space to discrete with scheduler selection and semi-dynamic isolation, implementing the complex three-branch reward function with terminal state logic, and building a custom DDQL agent with convolutional neural networks.

The most practical path forward depends on the project objectives. For full academic paper reproduction, the complete roadmap (Phases 1-5) with an estimated 3-4 weeks of effort is recommended. For a functional RL-based RAN slicing prototype with ns-3, the SAC alternative with the current continuous action space and a redesigned reward function (Phases 1 and 3 only) can be achieved in approximately 1-2 weeks. In either case, the ns-3 simulator provides the necessary data pipeline and control interface to support the DRL training loop, making the overall integration architecture sound even if individual components need adaptation.